DSA Explorer and Visualiser App

Project Report

(I completed the task alone, without any team members)

Item	Details
Student Name	Kaiwen Yang
Student ID	u3290623
Unit	Software Technology 2 (7170)
Teaching Period	Semester 1 2026
Assignment	ST2 Assignment 2 - Group Project
GitHub Repository	https://github.com/kaiwenyang895/DSA_Explorer_App

Contents

1. Introduction

This report presents the DSA Explorer and Visualiser App developed for Software Technology 2 (7170), Semester 1 2026. The purpose of the project is to help users understand important data structures and algorithms through interactive visualisation.

The project is modular, so each major part of the app is kept in its own Python file. This also makes the code easier to test and change.

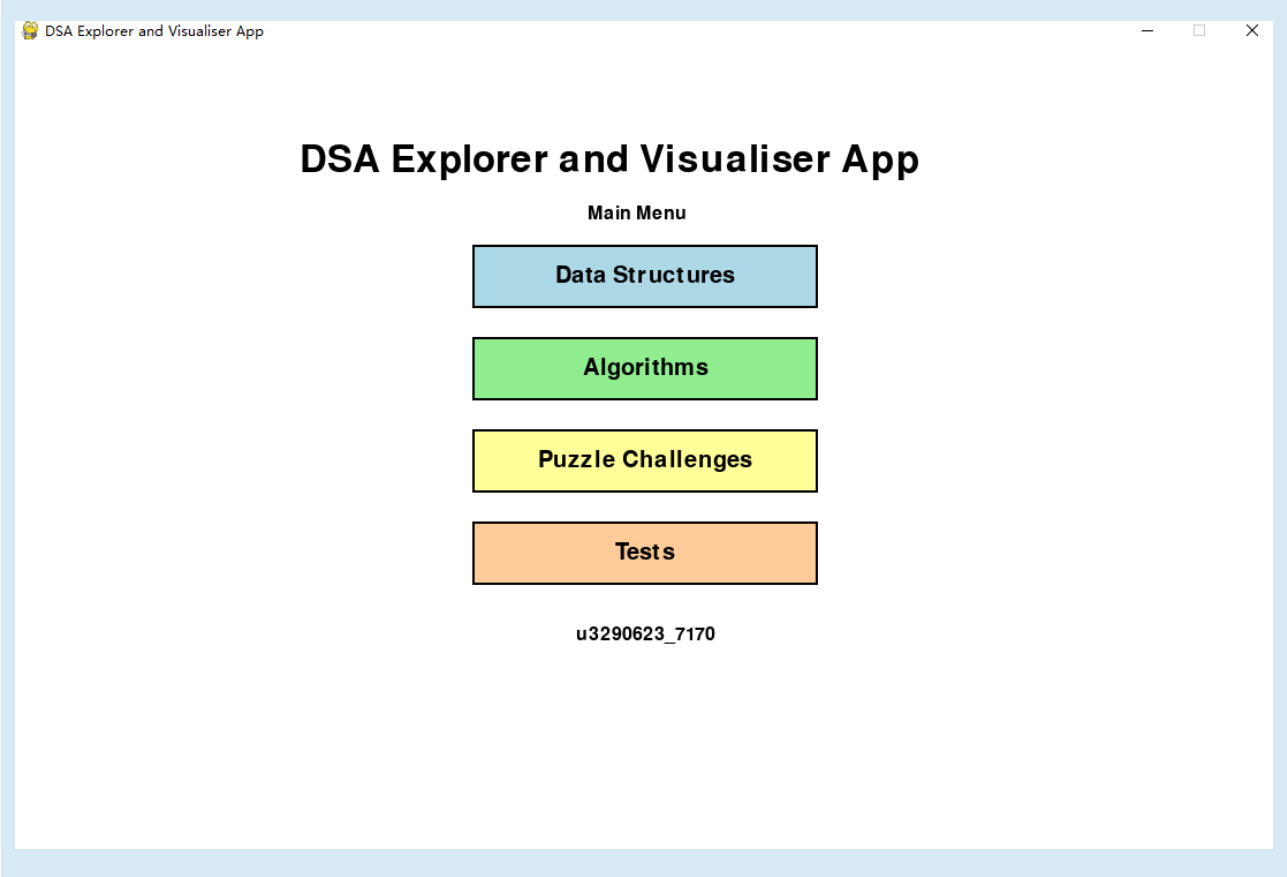

Figure 1. Main menu showing Data Structures, Algorithms, and Puzzle Challenges.

2. Project Requirements Mapping

Module	Implemented concepts
Data Structures	Stack with Push/Pop animation, Queue with Enqueue/Dequeue animation, Linked List with insert/delete/traverse/reverse, and Linear Search
Algorithms	Sorting Algorithms, BST Algorithms, Graph Traversals BFS/DFS, and Heap Priority Queue event simulator.
Puzzle Challenges	Puzzle module is kept as a separate app page. It can be used for pathfinding or other algorithm challenge screens.
Testing	Automated unit tests check linked list, sorting, BST, graph traversal, heap priority queue, linear search input, and simple benchmark results.

3. Data Structures Playground

The Data Structures module opens a sub-menu. From this page, the user can choose Stack, Queue, Linked List, or Linear Search.

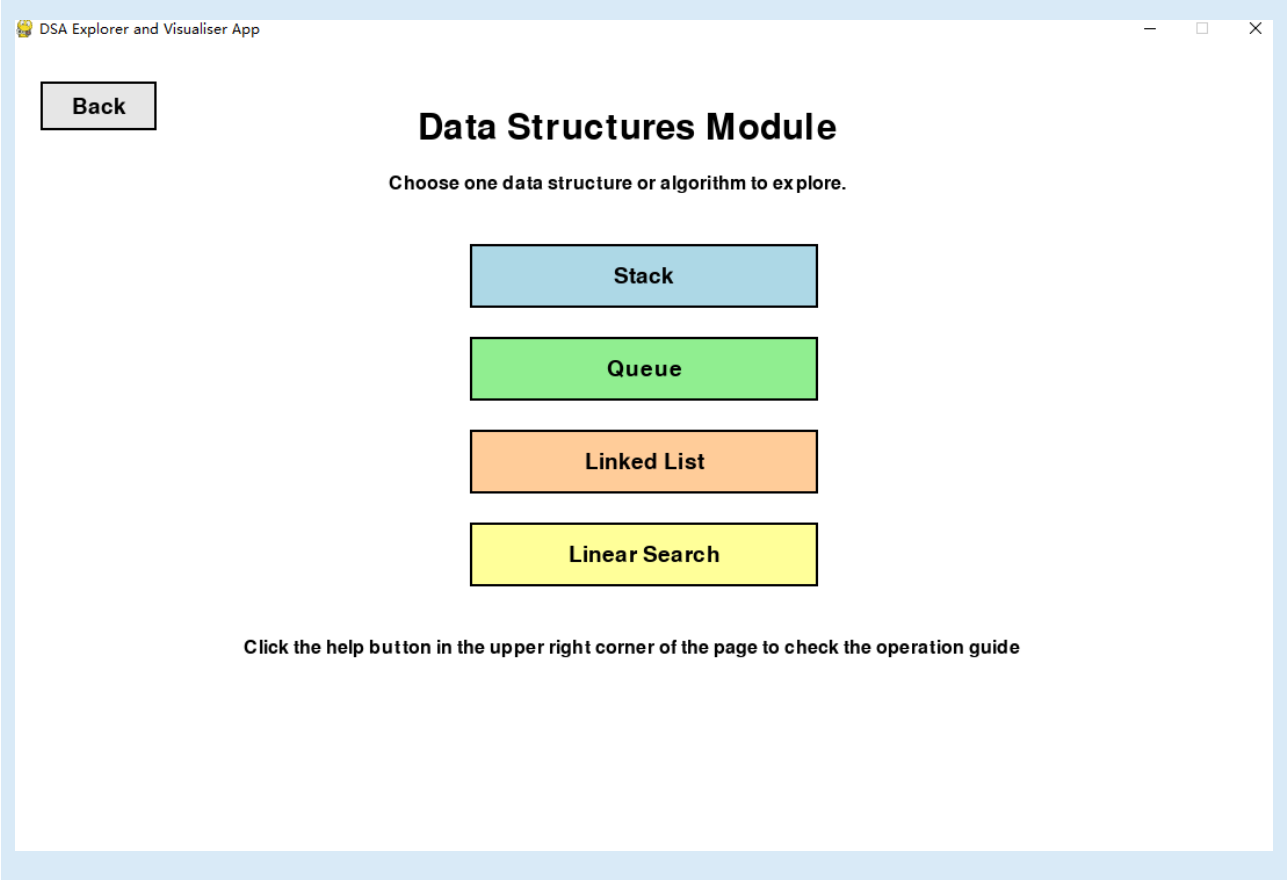

Figure 2. Data Structures menu with Stack, Queue, Linked List, and Linear Search buttons.

3.1 Stack Visualisation

The Stack page demonstrates the Last In, First Out (LIFO) principle.

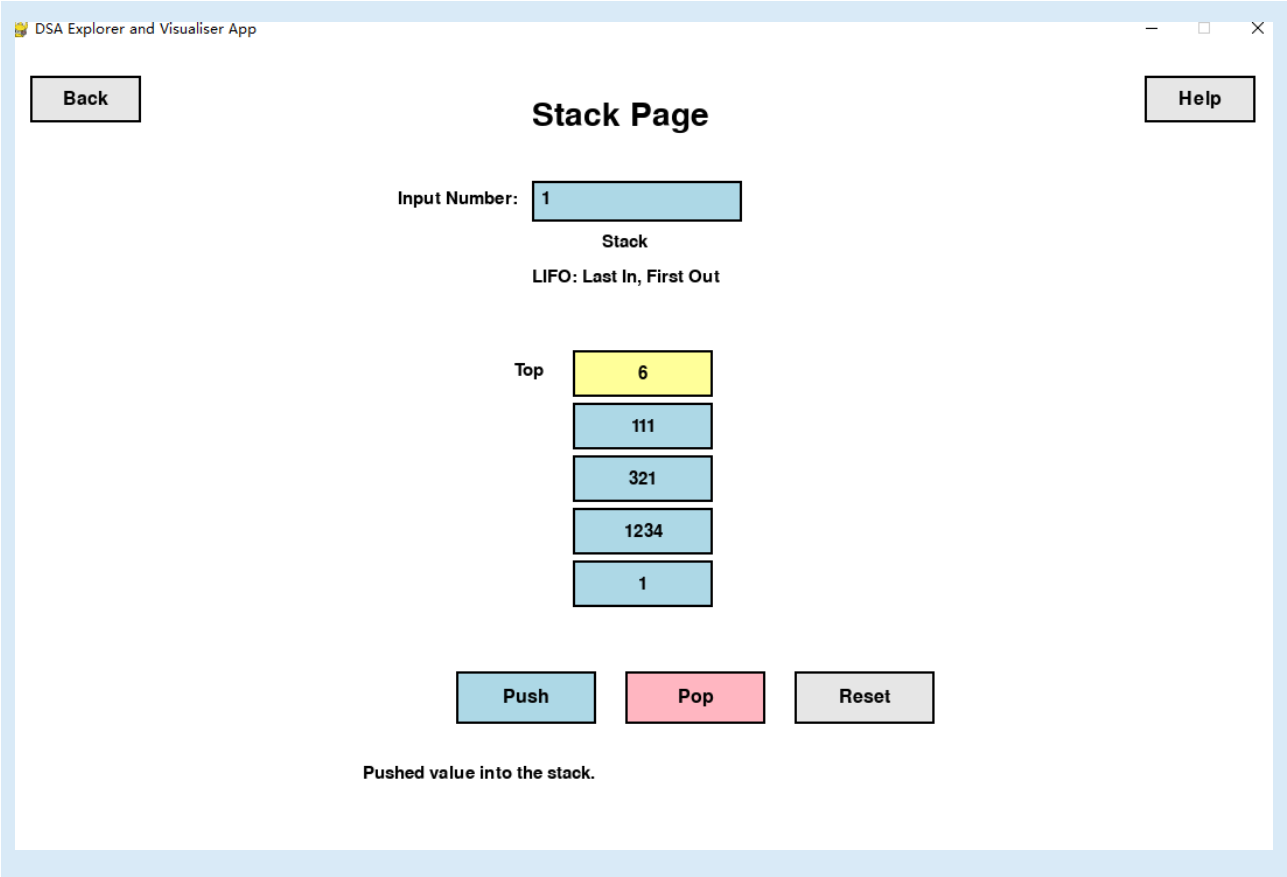

Figure 3. Stack page showing Push and Pop animation.

3.2 Queue Visualisation

The Queue page demonstrates the First In, First Out (FIFO) principle.

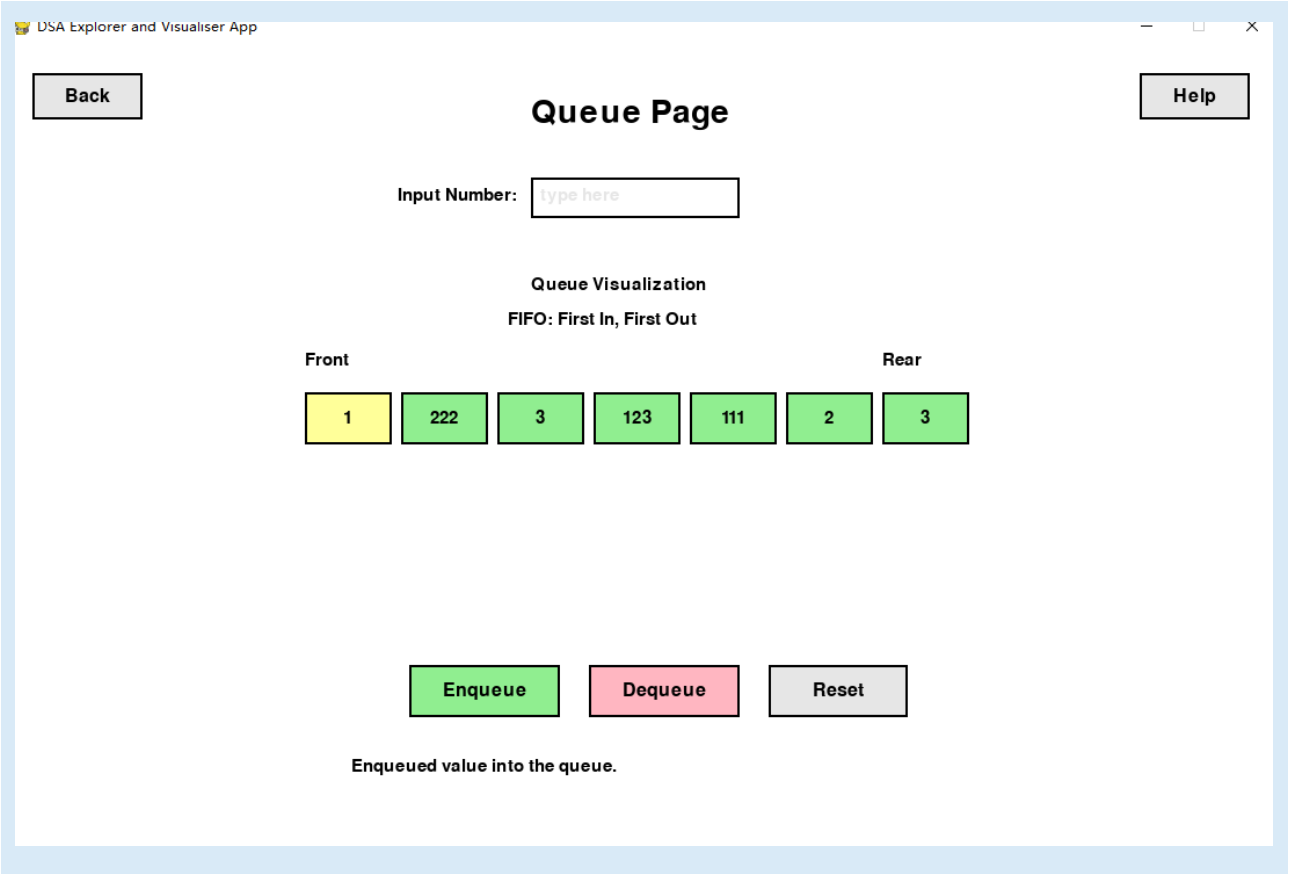

Figure 4 Queue page showing Enqueue and Dequeue sliding animation.

3.3 Linked List Visualisation

In the Linked List page, each node stores a value and a pointer to the next node.

The user can insert at the end, insert at a specified position, delete a value, traverse the list, and reverse the linked list.

The Reverse button shows the nodes move to the reversed positions.

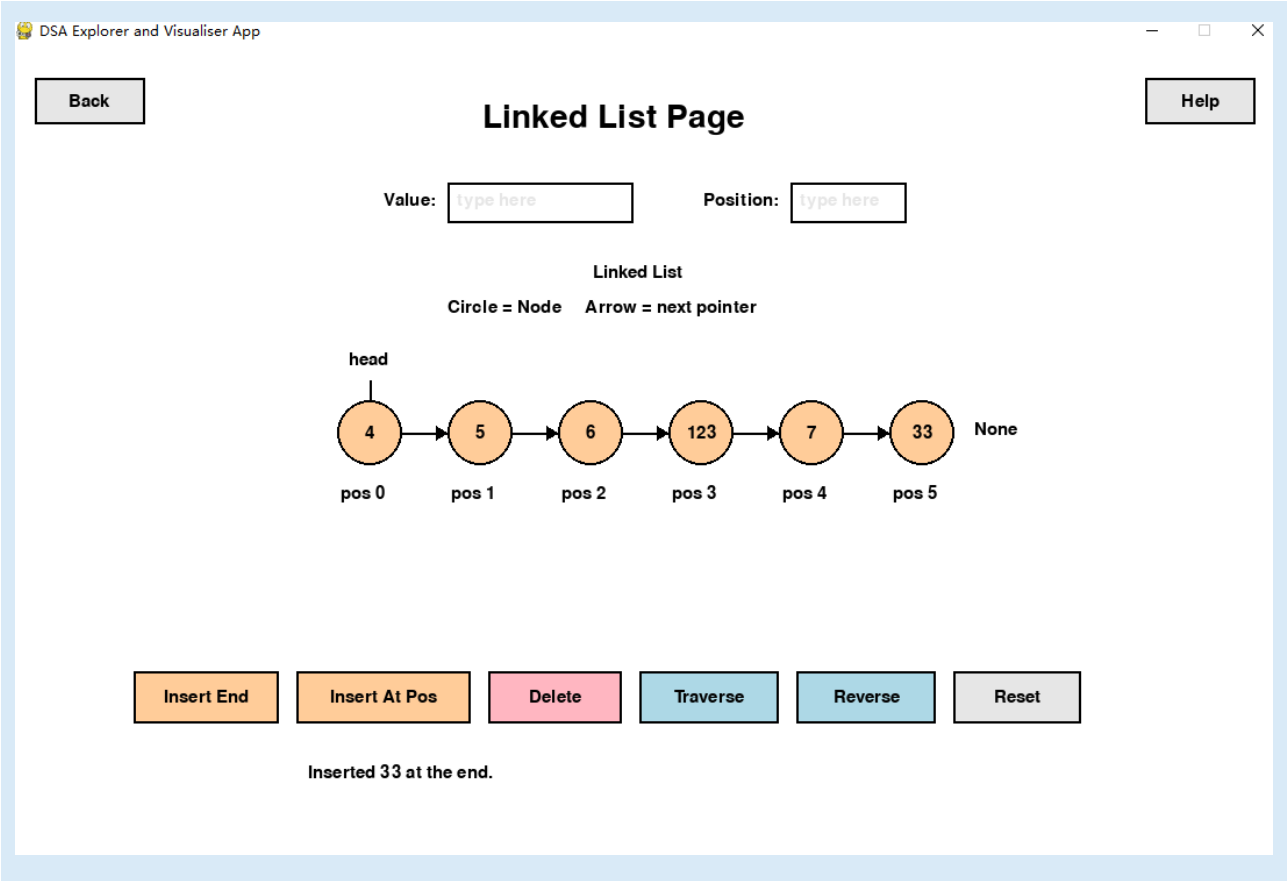

Figure 5. Linked List page showing circle nodes and next pointer arrows.

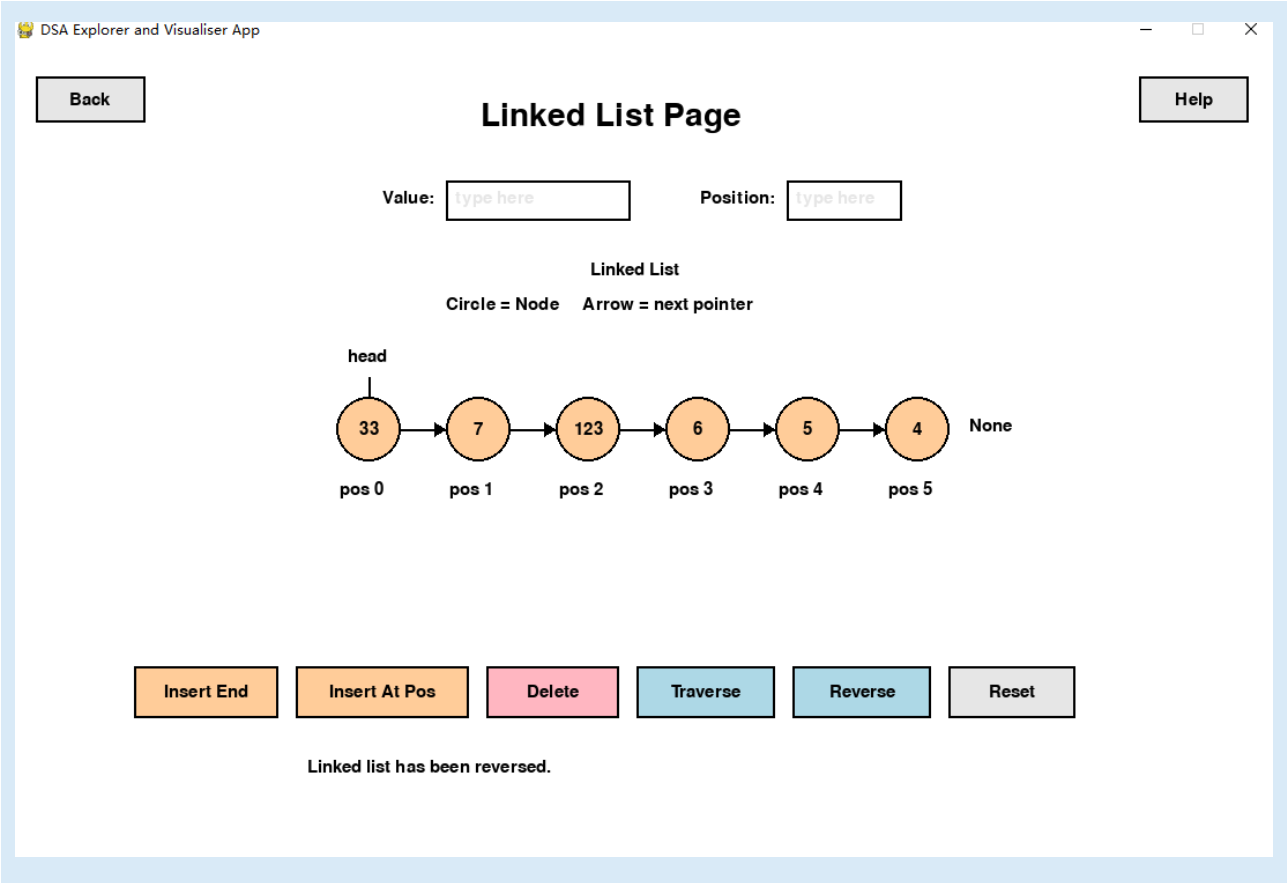

Figure 6. Linked List reverse

3.4 Linear Search Visualisation

The Linear Search page allows the user to type one or more target numbers. For example, the user can type 7 3 10. The program will search for each target one after another.

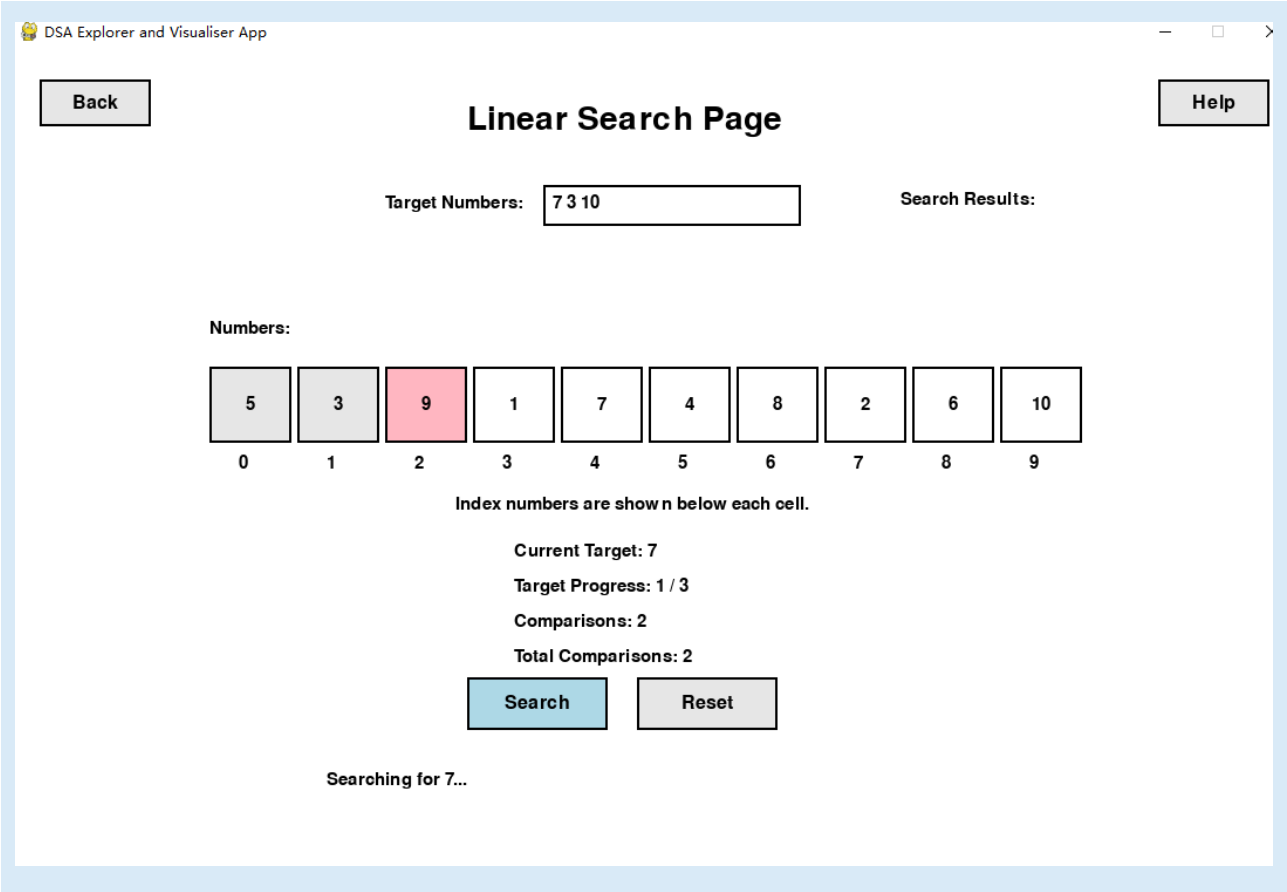

Figure 7. Linear Search page showing multiple targets and comparisons counter.

4. Phase 2 and Phase 3: Algorithms Module

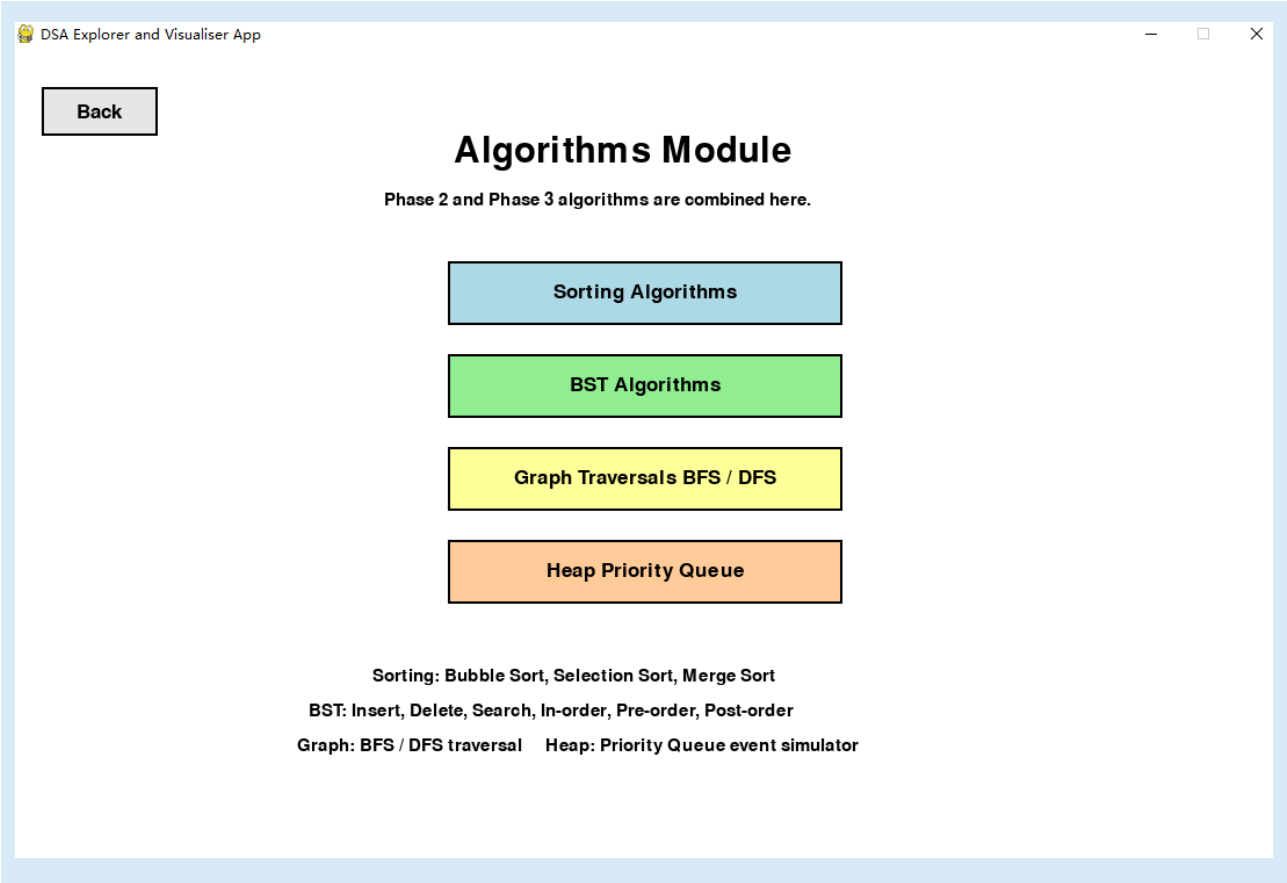

Figure 8. Algorithms menu showing Sorting, BST, Graph Traversals, and Heap Priority Queue.

4.1 Sorting Algorithms Module

Here I demonstrate how Bubble Sort, Selection Sort, and Merge Sort work. The bars represent the array. The height of each bar represents its value.

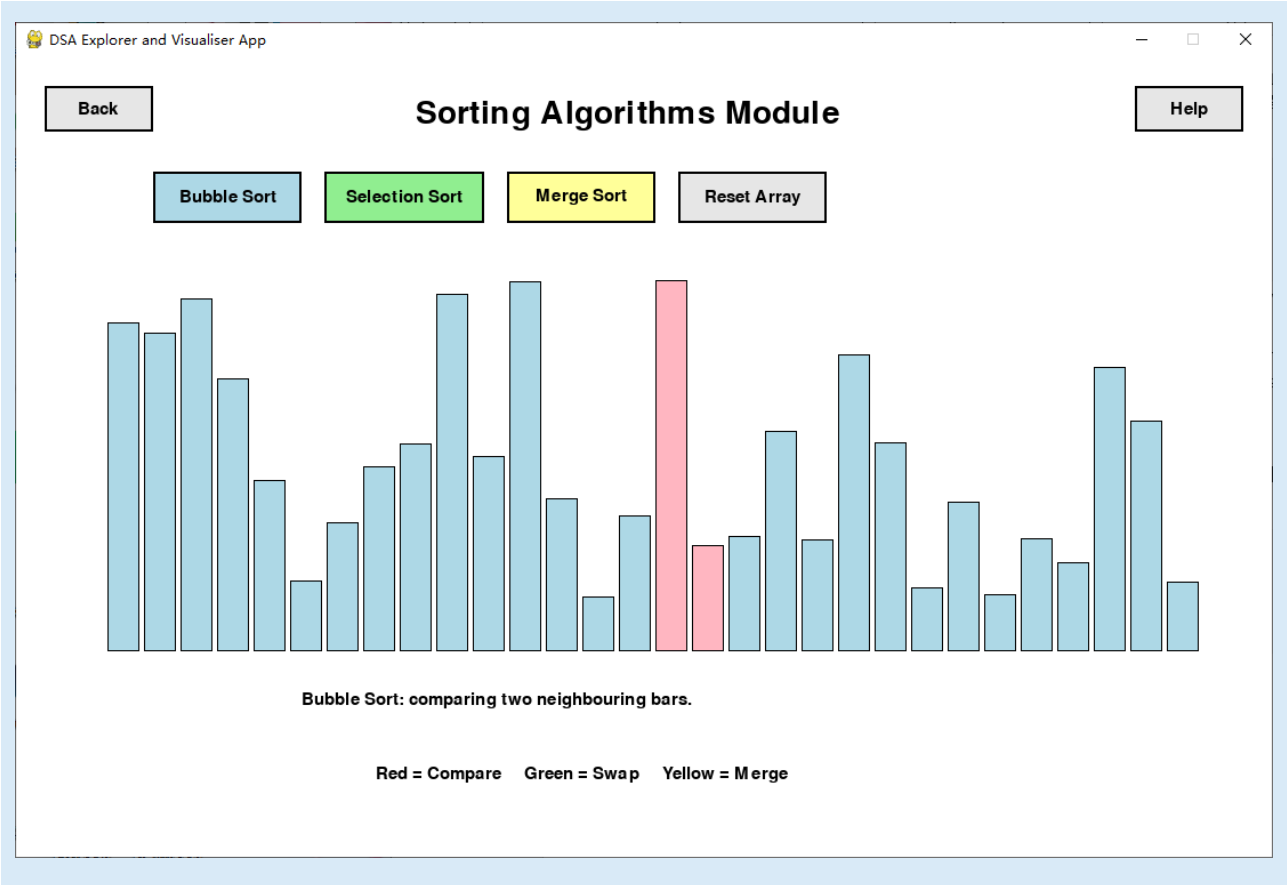

Figure 9. Sorting Algorithms module with Bubble, Selection, and Merge Sort buttons.

4.2 BST Algorithms Module

Here the BST Algorithms page visualises a Binary Search Tree. The user can insert a node, search for a node, delete a node, and run three traversal orders: in-order, pre-order, and post-order.

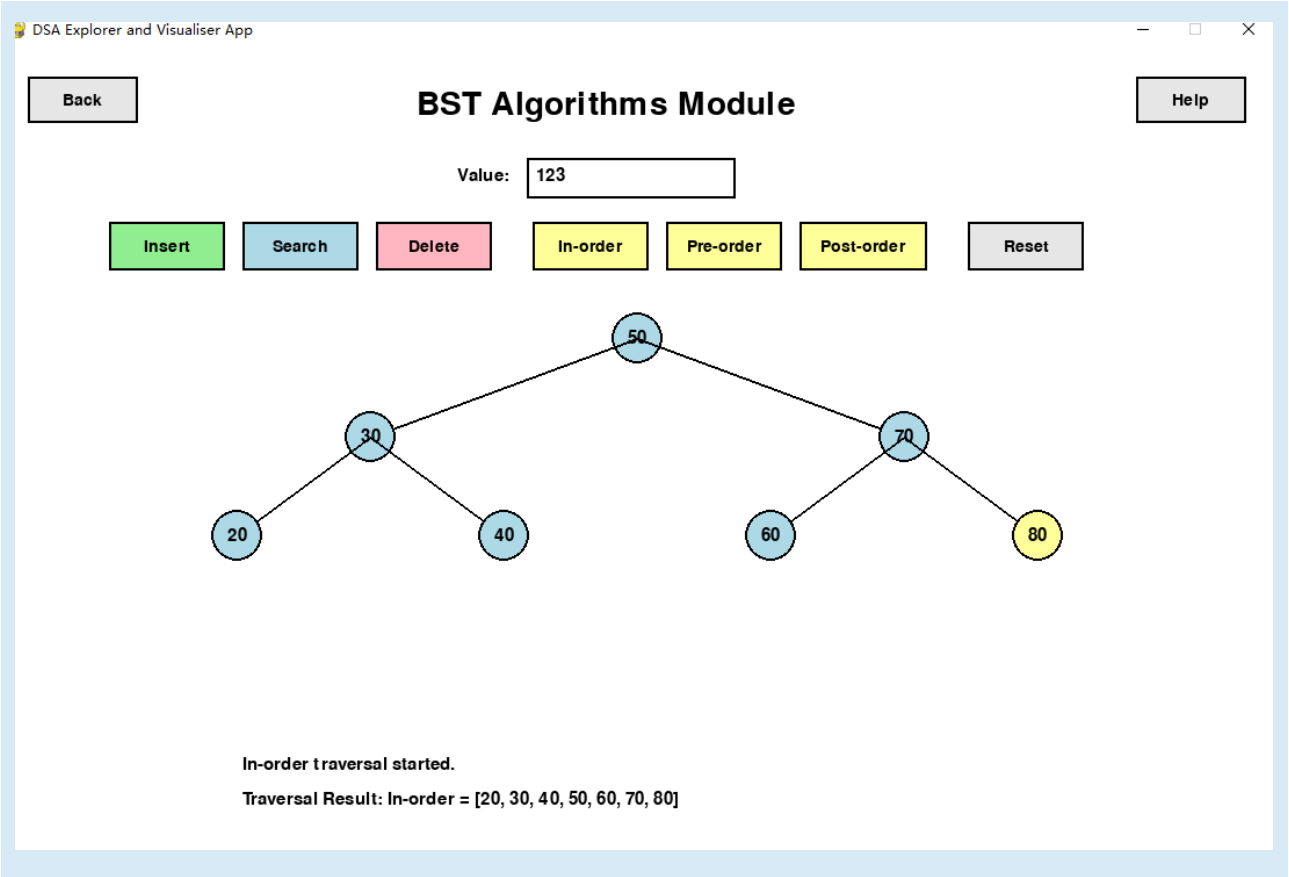

Figure 10. BST Algorithms module showing tree nodes and traversal buttons.

4.3 Graph Traversals BFS / DFS Module

The Graph Traversal module visualises Breadth First Search (BFS) and Depth First Search (DFS). I added a lot of visual animations.

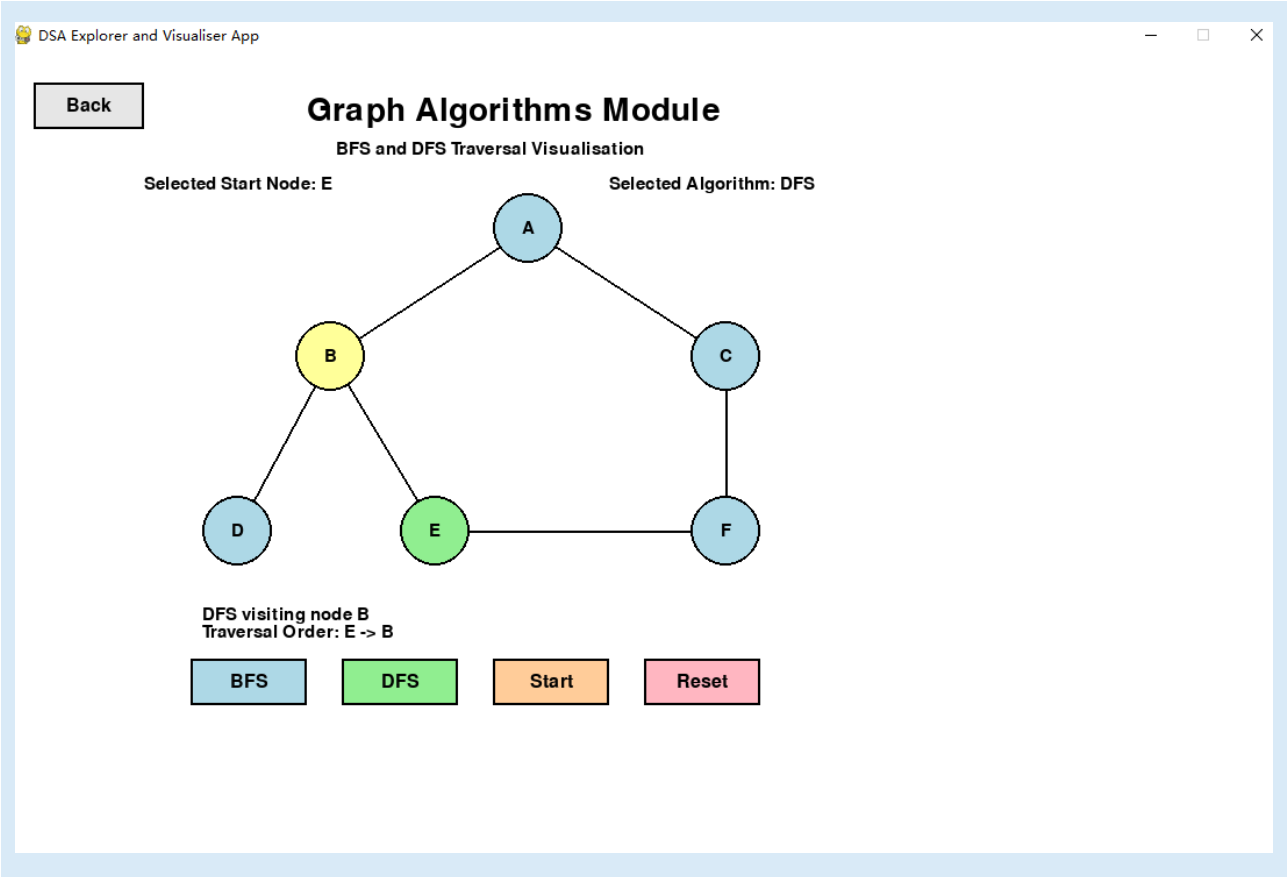

Figure 11. Graph Traversal module showing clickable nodes and traversal order.

4.4 Heap Priority Queue Module

I demonstrated the Heap Priority Queue module to simulate event scheduling. Users can freely enter the time and description.

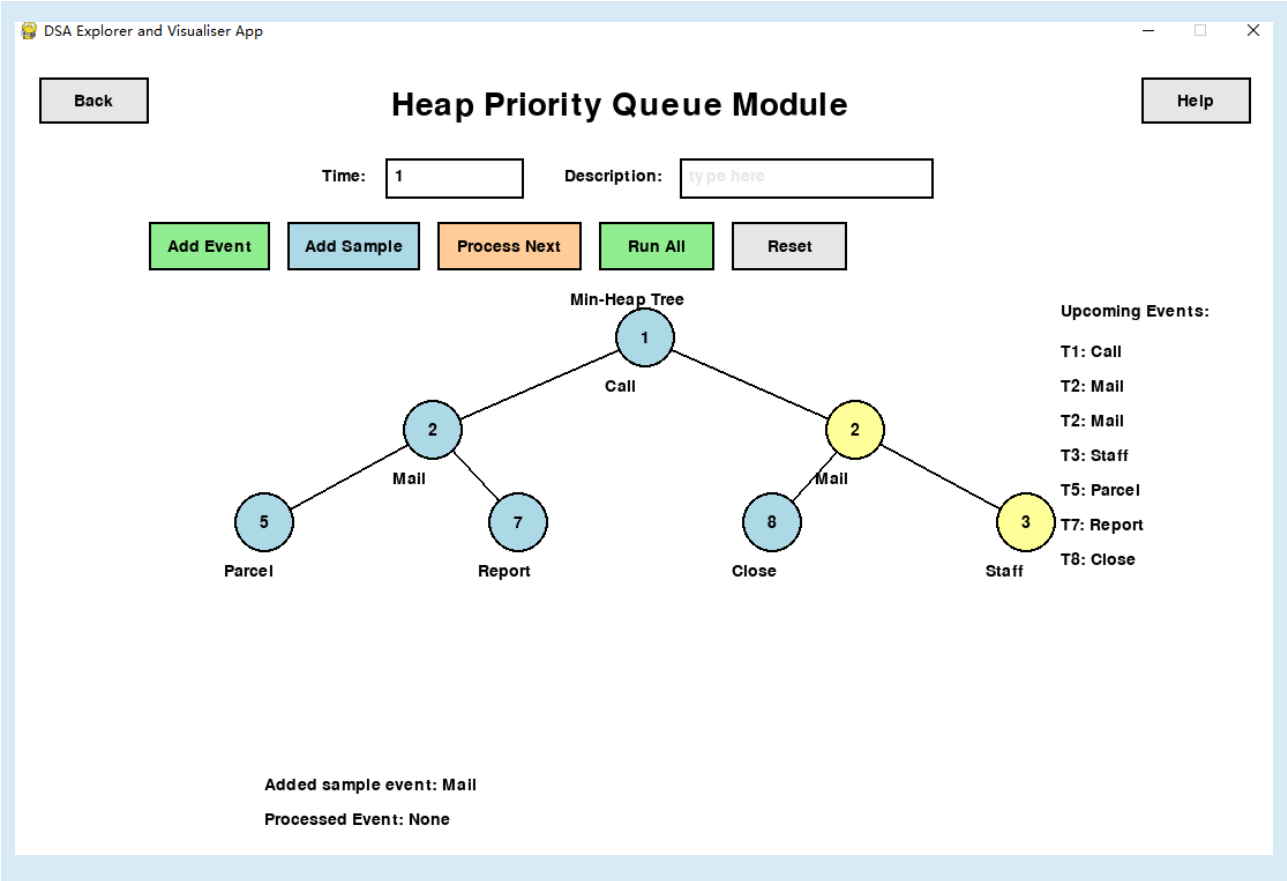

Figure 12. Heap Priority Queue module showing heap tree and upcoming events.

5. Puzzle Challenges Module

The Puzzle Challenges module contains a large amount of interactive content showcasing multiple concepts in the diagram.

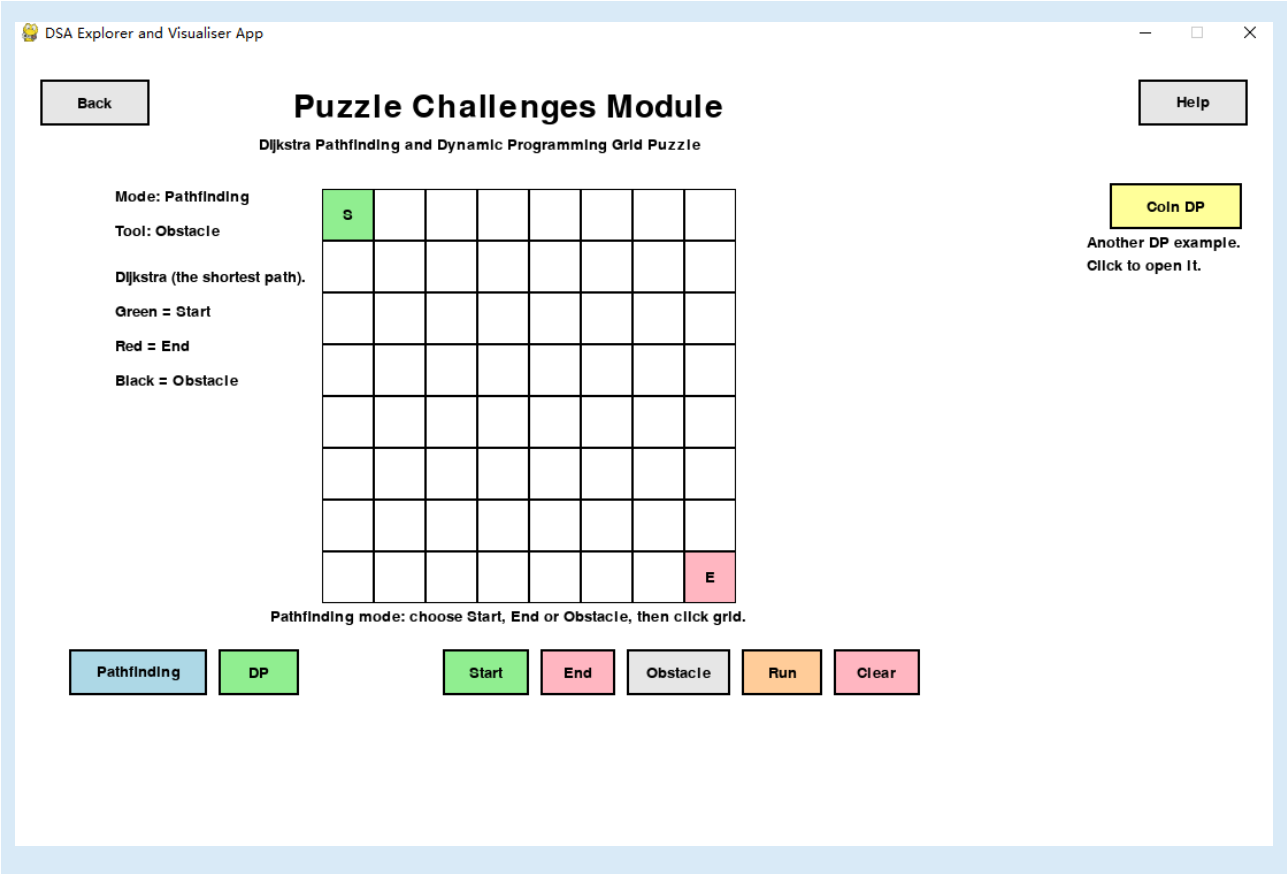

Figure 13. Puzzle Challenges module from the latest app version.

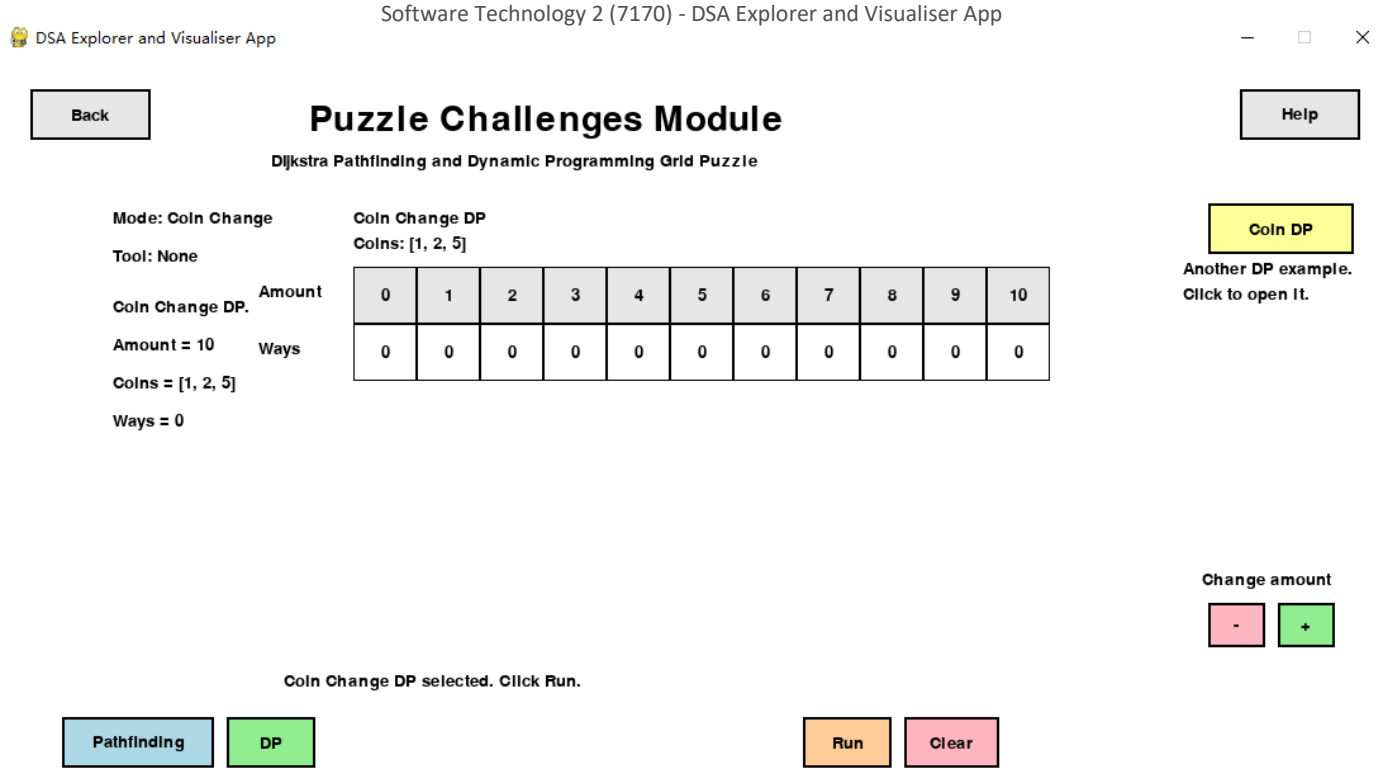

Figure 14. `coin` *DP*

6. Testing and Validation

Test Area	Validation Scenarios
Linked List	Insert at end, insert at position, delete value, reverse.
Sorting	Bubble Sort, Selection Sort, and Merge Sort final sorted result.
BST	In-order, pre-order, post-order, search path, delete leaf node, delete two-child node.
Graph Traversal	BFS and DFS order from start node, and checking all reachable nodes are visited.
Heap Priority Queue	Insert events, extract earliest event, and empty extract-min.
Linear Search	Parse multiple target values using spaces or commas.
Benchmarking	Small runtime checks for sorting and heap insert/extract operations.

6.1 Automated Test Result

Figure 15. Automated testing screenshot showing test result and benchmark output.

7. Help and Instructions

Most pages contain a help button to assist users in understanding how to operate.

8. Future Improvements

Currently, the overall display of the animation is not smooth enough, and the button layout is not aesthetically pleasing. In the future, I will modify the animation so that it can be accelerated. I will redraw the buttons.

9. Conclusion

Overall, this project has all the main application functional requirements, including data structures, sorting, BST, graph traversal, heap priority queue, and a puzzle challenge page. The project fully meets a modular, interactive layout, and also includes automated testing and benchmarking.

References

Pygame Community. (n.d.). Pygame documentation. https://www.pygame.org/docs/

Python Software Foundation. (n.d.). unittest — Unit testing framework. Python documentation. https://docs.python.org/3/library/unittest.html